Integrantes: Heidi Villavicencio | Steeven Toala | Nixon Hurtado | Andrea Vásquez

1. TÍTULO

Denominación del Entregable: Integración del Editor Enriquecido TipTap con Autoguardado Asíncrono Debounce (1.5s) en React + Vite hacia API REST Express v5

2. INTRODUCCIÓN

En la elaboración de planes de trabajo e informes de cumplimiento, las secciones descriptivas (Antecedentes, Justificación, Metodología de Cátedra, Conclusiones y Recomendaciones) demandan una redacción fluida con soporte para negritas, listas ordenadas, viñetas y formato estructurado conforme a la norma ISO 9001:2015 del SGC. El empleo de cajas de texto planas (`textarea`) degrada la experiencia de usuario y no genera la semántica enriquecida requerida para la posterior exportación documental.

Para subsanar esta limitación, se integró el motor de edición enriquecida **TipTap** (construido sobre el estándar *ProseMirror*) en el cliente **React + Vite**. Para blindar al docente contra cualquier pérdida involuntaria de información por cortes de fluido eléctrico o intermitencias de red en el campus, se implementó un mecanismo de **autoguardado asíncrono en segundo plano mediante un Hook personalizado** `useAutoSaveSection` **con Debounce calibrado a 1500 ms**, informando en tiempo real el estado de sincronización hacia el backend Node.js + Express (v5) y la base de datos PostgreSQL (18).

3. OBJETIVO

3.1. Objetivo General

Implementar el componente de edición enriquecida `RichTextEditor.jsx` en React y Vite basado en TipTap, dotándolo de herramientas de formato tipográfico institucional, sanitización XSS y sincronización asíncrona automatizada hacia la API REST de Express v5.

3.2. Objetivos Específicos

- Configurar TipTap con extensiones esenciales: `StarterKit`, `Bold`, `Italic`, `BulletList`, `OrderedList`, `Heading` e `History (Undo/Redo)`.
- Diseñar el Hook reactivo `useAutoSaveSection` para emitir solicitudes HTTP `PUT /api/documentos/:id` únicamente tras 1.5 segundos de inactividad del teclado, evitando sobrecarga innecesaria en el servidor.
- Implementar badges visuales reactivos en la interfaz: `[Guardando...]`, `[Guardado en PostgreSQL]` o `[Error de red - Reintentando]`.

- Sanitizar el contenido HTML generado mediante `DOMPurify` para neutralizar riesgos de inyección de código malicioso (XSS).

4. ALCANCE

4.1. Alcance Incluido

Componente `RichTextEditor.jsx` modular y reutilizable para cualquier sección del expediente, Hook de autoguardado asíncrono con control de cancelación (`AbortController`), sanitización con DOMPurify y persistencia asíncrona sin bloquear la escritura en el cliente.

4.2. Límites y Exclusiones

No incluye corrección gramatical asistida por IA ni edición colaborativa simultánea en tiempo real vía WebSockets.

5. DEFINICIONES

- **TipTap:** Framework moderno de edición enriquecida desacoplado del DOM nativo y estructurado sobre ProseMirror.
- **Debounce:** Técnica de optimización que posterga la ejecución de una rutina asíncrona costosa hasta que transcurre un período de inactividad.
- **DOMPurify:** Biblioteca de sanitización de código HTML contra inyecciones XSS.
- **EDT (*Estructura de Desglose del Trabajo*):** Descomposición jerárquica de entregables del proyecto.

6. RESPONSABILIDADES

- **Andrea Vasquez (Frontend Developer / QA):** Programación de componentes TipTap, barra de herramientas y Hook debounce.
- **Steeven Toala (Líder / Arquitecto):** Optimización de contratos API REST, pruebas de rendimiento de guardado y sanitización XSS.
- **Ing. Paulo Torres (Docente Tutor):** Supervisión técnica y validación del comportamiento de autoguardado en la experiencia docente.

7. NORMATIVA LEGAL Y TÉCNICA

- **OWASP ASVS 4.0 (Sección V5):** Validación, sanitización y codificación de entradas en editores WYSIWYG contra ataques XSS.
- **ISO/IEC 25010:** Eficiencia de desempeño, tolerancia a fallos y confiabilidad de guardado.

- **Manual de Procedimientos SGC FISEI UTA:** Estándares de presentación de informes de cumplimiento académico.

8. MÉTODO, POLÍTICAS Y CONTENIDOS

Desarrollo basado en componentes funcionales de React con el cliente Axios institucional con auto-renovación silenciosa de tokens JWT. Se establecen políticas rigurosas:

1. **Ventana de Debounce:** La solicitud de persistencia se dispara tras 1500 ms de pausa continua en la digitación.
2. **Cancelación Preventiva:** Si el usuario reanuda la escritura mientras existe una petición pendiente, se cancela vía `AbortController`.
3. **Respaldo Local:** En caso de corte prolongado, el texto se preserva en `localStorage` como contingencia.

9. DESCRIPCIÓN DE ACTIVIDADES

9.1. Hook de Autoguardado Asíncrono con Debounce

Se programó el hook `useAutoSaveSection` que posterga la petición HTTP hasta que transcurran 1500 ms de inactividad:

```
// frontend/src/hooks/useAutoSaveSection.js
import { useState, useEffect, useRef } from 'react';
import api from '../api/client';

export function useAutoSaveSection(docId, claveSeccion, contenido) {
  const [saveStatus, setSaveStatus] = useState('saved');
  const abortRef = useRef(null);

  useEffect(() => {
    if (!docId || !claveSeccion) return;
    setSaveStatus('saving');
    if (abortRef.current) abortRef.current.abort();
    abortRef.current = new AbortController();

    const timer = setTimeout(async () => {
      try {
        await api.put(`/documentos/${docId}/secciones`, {
          claveSeccion,
          contenidoTexto: contenido
        }, { signal: abortRef.current.signal });
        setSaveStatus('saved');
      } catch (err) {
```

```
        if (err.name !== 'CanceledError') setSaveStatus('error');
      }
    }, 1500);

    return () => clearTimeout(timer);
  }, [contenido, docId, claveSeccion]);

  return { saveStatus };
}
```

9.2. Extensiones TipTap y Sanitización DOMPurify

Se configuró el conjunto de extensiones `StarterKit` para dar soporte a títulos jerárquicos, listas ordenadas y citas. Todo contenido generado pasa por un filtro estricto de sanitización con `DOMPurify` para neutralizar posibles vectores XSS.

9.3. Insignias Visuales de Estado

La interfaz refleja en tiempo real el ciclo de persistencia mediante badges accesibles: `[Guardando...]` en amarillo ámbar, `[Guardado en PostgreSQL]` en verde esmeralda institucional y `[Error de red]` en rojo con reintento asistido.

9.4. Conclusiones y Recomendaciones Técnicas

- La integración de TipTap con debounce de 1.5s garantiza que el docente nunca pierda su trabajo por descuidos de guardado manual.
- El formateo semántico generado es limpio y compatible con los motores de renderizado PDF/Word.
- Mantener el almacenamiento de copias locales en `localStorage` como respaldo de contingencia ante caídas de conexión prolongadas.

10. FIRMAS DE RESPONSABILIDAD

ACCIÓN	NOMBRE Y CARGO	FIRMA	FECHA
Elaborado por:	Heidi Villavicencio Analista de Requisitos / UI-UX		25/09/2026
Revisado por:	Steeven Toala Líder de Proyecto / Arquitecto		25/09/2026
Revisado por:	Nixon Hurtado Desarrollador Backend / DBA	**[FIRMADO ELECTRÓNICAMENTE]**	25/09/2026
Revisado por:	Andrea Vásquez Desarrolladora Frontend / QA	**[FIRMADO ELECTRÓNICAMENTE]**	25/09/2026
Aprobado por:	Ing. Paulo Torres, Mg. Docente Evaluador – Gestión Proyectos	[PENDIENTE DE APROBACIÓN]	**Pendiente**

11. CONTROL DE CAMBIOS

Versión	Descripción del cambio	Responsable del cambio	Fecha de aprobación
1.0	Integración inicial del editor TipTap y hook de autoguardado.	Andrea Vasquez	16/09/2026
2.0	Actualización al stack oficial: React + Vite, Tailwind CSS #00345E, cliente Axios con auto-renovación silenciosa y persistencia en backend Express v5 + PostgreSQL 18.	Andrea Vasquez / Steeven Toala	16/09/2026

12. ANEXOS

Anexo 1: Referencias Bibliográficas

- TipTap Core Team. (2026). *TipTap Documentation: Modern Rich Text Editing*.
- ProseMirror Guide. (2024). *Document Model and State Transitions*.
- OWASP Foundation. (2023). *Cross-Site Scripting Prevention Cheat Sheet*.

Anexo 2: Manejo de Errores y Estrategia de Reintento

Mecanismo de respaldo local en caso de cortes de servicio web antes de la sincronización definitiva en PostgreSQL 18.